Engineering Mini-Challenge: Intelligent Freight Load Board & Autonomous Dispatch Engine

1. Executive Summary & Background

In the freight transportation and logistics industry, carriers manage hundreds of regional shipments across multiple geographic hubs (terminals) every day. Freight dispatchers monitor incoming shipment opportunities ("loads") published on customer portals. These opportunities generally arrive in two formats:

- **Fixed-Rate Offers ("Book It Now"):** The customer offers a predetermined, non-negotiable price. The dispatcher must quickly evaluate whether the payout covers operational costs (fuel, wear-and-tear, driver pay) and accept or decline before a competing carrier takes it.
- **Open Bids:** The customer posts lane details (origin, destination, weight, pickup time) without a set price. Carriers must submit a competitive bid rate ($) to secure the freight while maintaining target profit margins.

Why This Matters

- **The Business Problem:** Dispatchers make high-volume, high-stress, and time-sensitive decisions. They must navigate hard physical constraints, such as managing a finite number of tractors and trailers at each terminal, weight limits, and geographic alignment. Furthermore:
    - **Deadheading:** Driving a truck with an empty trailer to a pickup location burns fuel and wastes money.
    - **Resource Scheduling (Time Windows):** Every order has a strict pickup time, drop-off time, and Estimated Time of Arrival (ETA). Once a truck and driver are assigned an order, they are locked out and unavailable for any other work during that exact time window.
- **The Engineering Challenge:** You are tasked with building a full-stack, real-time web application and decision-support engine that processes historical data and fleet capacity inventories, consumes a live simulated API feed of freight orders, tracks temporal resource allocation, optimizes routes to minimize empty miles, and automates booking logic.

2. Provided Resources, Data Streams & Fleet Constraints

To jumpstart your development, we provide the following assets:

- **Historical Dataset:** Access to past shipment records, containing historical pricing, lane data, and win/loss outcomes to analyze or train predictive models.
- **Fleet Capacity Inventory:** Initial baseline data detailing the exact number of available trucks, trailers, and drivers assigned to each terminal.
- **Live Simulated API Feed:** A constant stream of real-time "fake" orders simulating live customer traffic. Orders appear dynamically with ticking expiration countdowns via WebSockets or polling.
- **Order Attributes & Constraints:** Every incoming order specifies origin, destination, weight, equipment type, payout rate, expiration window, and strict pickup/drop-off time constraints (ETAs).

3. Core System Requirements

Your solution must integrate several classic software engineering, scheduling, and distributed systems components:

A. Relational Database & State Management

- Design a relational schema to track shipment attributes, real-time terminal inventory, and temporal resource locks (tracking when specific trucks/drivers are busy vs. idle based on order time windows).

B. Real-Time Data Ingestion Pipeline

- Consume the provided live API feed of incoming load opportunities and dynamic expiration countdowns, updating system state concurrently.

C. Fleet, Schedule & Route Optimization Engine

- **Time-Window Conflict Checking:** Prevent double-booking: a truck/driver cannot be assigned to overlapping order time windows.
- **Empty-Mile Reduction (Deadheading):** Implement logic/algorithms that help dispatchers or the automated system chain loads together (e.g., dropping off Load A and immediately picking up Load B nearby) to minimize empty trailer mileage.
- **Mileage Optimization:** Maximize operational efficiency by prioritizing orders that yield optimal total mileage and route continuity for each truck.

D. Web-Based Dispatch Interface (Frontend)

Build a responsive frontend UI allowing dispatchers to:

- View live-updating freight queues filtered by Fixed-Rate vs. Open-Bid status.
- Accept/decline fixed-rate shipments with immediate UI feedback.
- Input and submit custom rate bids for open-bid shipments.
- Monitor real-time terminal truck/driver utilization dashboards, time-schedule timelines, and capacity warnings.

E. Business & Constraint Logic (Backend Enforcement)

- Enforce strict backend guardrails that prevent orders from being accepted if a terminal lacks available trucks/drivers *during that specific time window*, or if weight/location constraints are violated.

4. Optional Advanced Extensions

If you want to push your software architecture, algorithms, and machine learning skills further, implement one or both of the following:

A. **Machine Learning / Predictive Analytics:** Train an AI model using the *provided historical data* to evaluate lane profitability, predict bid win probabilities, and suggest optimal rate markups.
B. **Autonomous "Auto-Pilot" Dispatch Mode:** Build a background worker service that automatically accepts high-margin fixed loads and submits competitive bids for open freight without human intervention—strictly bounded by terminal capacity caps, time-window availability, empty-mile optimization, and safety thresholds.

5. Suggested Deliverables

A. **Architecture Documentation:** Database Entity-Relationship Diagram (ERD), system architecture diagram, and data flow explanations (specifically highlighting how time-window locking is handled).
B. **Backend Services & APIs:** Functional backend service handling API feed ingestion, capacity tracking, temporal constraint validation, and routing/optimization algorithms, exposed via REST/WebSocket endpoints.
C. **Frontend Prototype:** An interactive dispatcher web application demonstrating the live workflow, fleet utilization, and schedule tracking.
D. **Engineering & Business Report:** A concise summary outlining how business constraint logic, time-window scheduling, and empty-mile optimization algorithms were implemented, how historical data was leveraged, and recommendations for scaling the system to an enterprise level.